

CRUX: HOW TO STORE AND ACCESS DATA ON DISK

How do modern hard-disk drives store data? What is the interface? How is the data actually laid out and accessed? How does disk scheduling improve performance?

The interface

The drive consists of a large no. of sectors (512-byte blocks), each of which can be read or written.

The sectors are numbered 0 to $n-1$ on disk with n sectors. This is the address space of the disk.

Many file systems will read or write 4KB or more at a time (multi-sector operations). However,

It is important to note that drive manufacturers only guarantee that a single 512-byte write is atomic.
(sector)

Thus, if an untimely power loss occurs, only portion of a larger write may complete called a torn write.

Clients of disk drivers make below assumptions:

- Accessing two sectors near one another within the drive's address space will be faster than accessing two sectors that are far apart.
- Accessing sectors in a contiguous chunk (sequentially) is the fastest access mode & much faster than any random access pattern.

Basic Geometry

A modern disk consists of:

Platter, A circular hard surface on which data is stored persistently by inducing magnetic changes to it.

A disk may have one or more platters; each platter has

2 sides; each of which is called a surface.

These platters are made of aluminum, and then coated with a thin magnetic layer that enables them to store data persistently.

The platters are all bound together around a spindle, which is connected to a motor that spins the platters at a constant rate.

The rate of rotation is measured in rotations per minute (RPM).

We are often interested in the time of single rotation.
for example,

A drive that rotates at 10,000 RPM means that a single rotation takes 6 milliseconds.

Data is encoded on each surface in concentric circles of sectors called tracks. A single surface contains thousands of tracks tightly packed together.

A disk head either senses (read) magnetic patterns on the disk or induces a change (write) in them.

The disk head is attached to the disk arm, which moves across the surface to position the head over desired tracks.



A simple Disk Drive

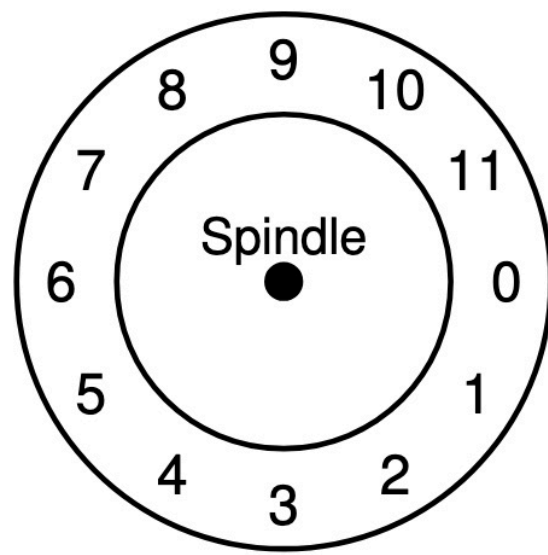


Figure 37.1: A Disk With Just A Single Track

Assume we have a simple disk with a single track as above. This track has 12 512-byte sectors and addressed by numbers 0-11.

The single platter rotates around the spindle, to which a motor is attached.

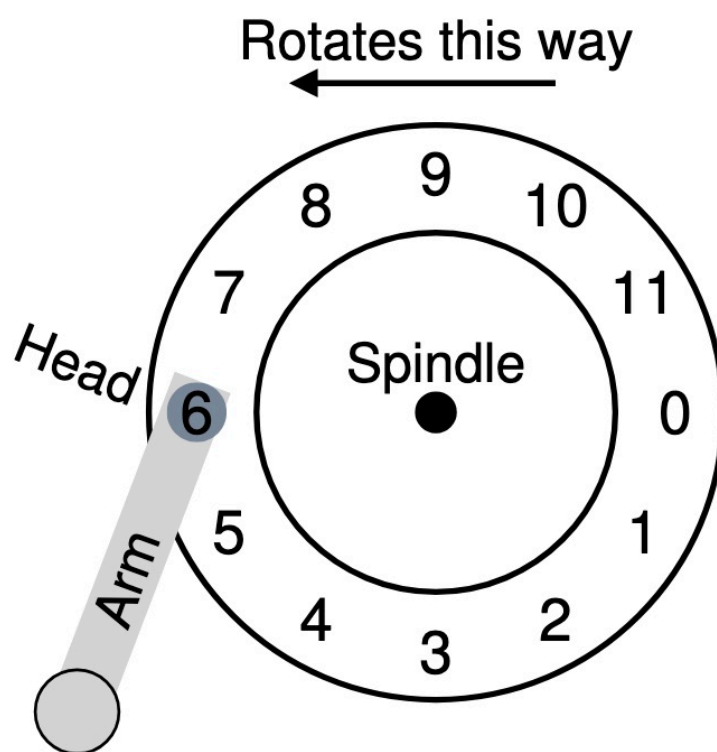


Figure 37.2: A Single Track Plus A Head

To be able to read or write, we need a disk head attached to a disk arm like above.

The surface rotates counter clockwise and the disk head is positioned over sector 6.

Single-track Latency: The Rotational Delay

Imagine we receive a request to read sector 0.

Our simple disk will simply wait for the desired sector to rotate under the disk head.

This wait is called rotational delay.

In our example, if full rotational delay is R , the disk has to incur a rotational delay of $\frac{R}{2}$.

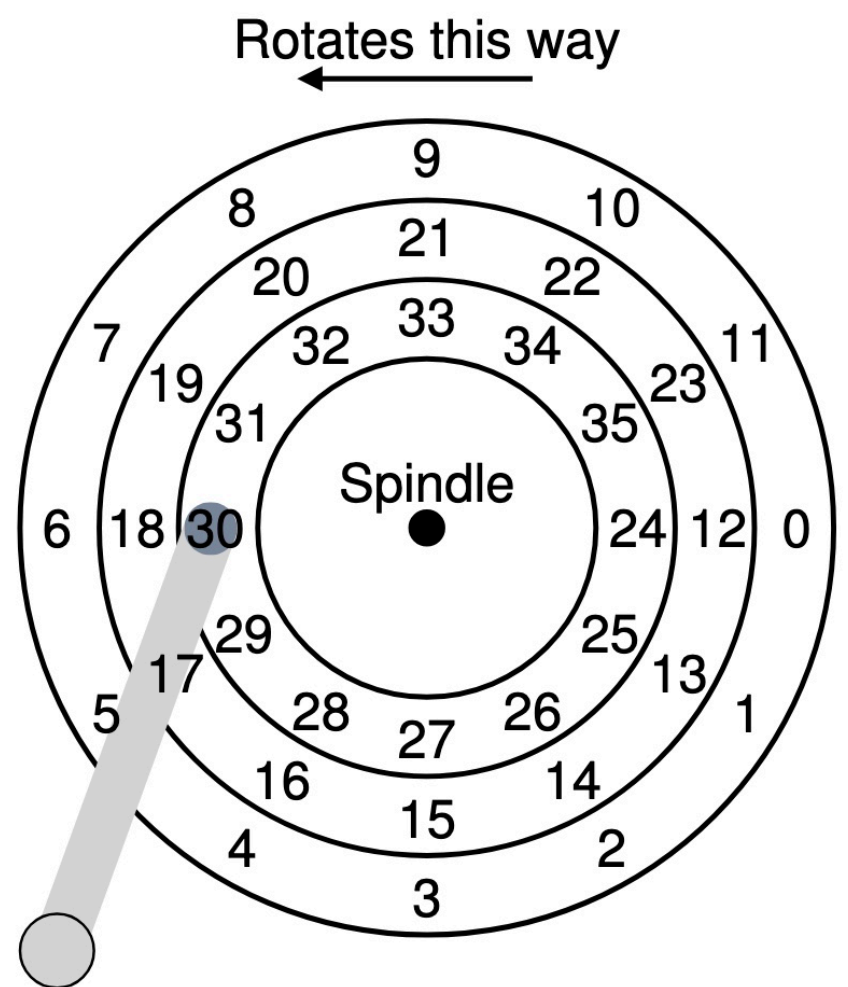
Multiple Tracks: Seek time

In right figure, the head is positioned over the inner-most track.

Imagine a read request of sector 11.

To service this request, the drive has to first move the disk arm to the outermost track, this process is called seek.

Seek along with rotations are wstly disk operations.



A more realistic model

The seek has many phases:

1. Acceleration: The disk arm gets moving
2. Coasting: The arm is moving at full speed
3. Deceleration: The arm slows down.
4. Settling: The head is carefully position over the correct track.

The settling time is quite significant from 0.5ms to 2ms.

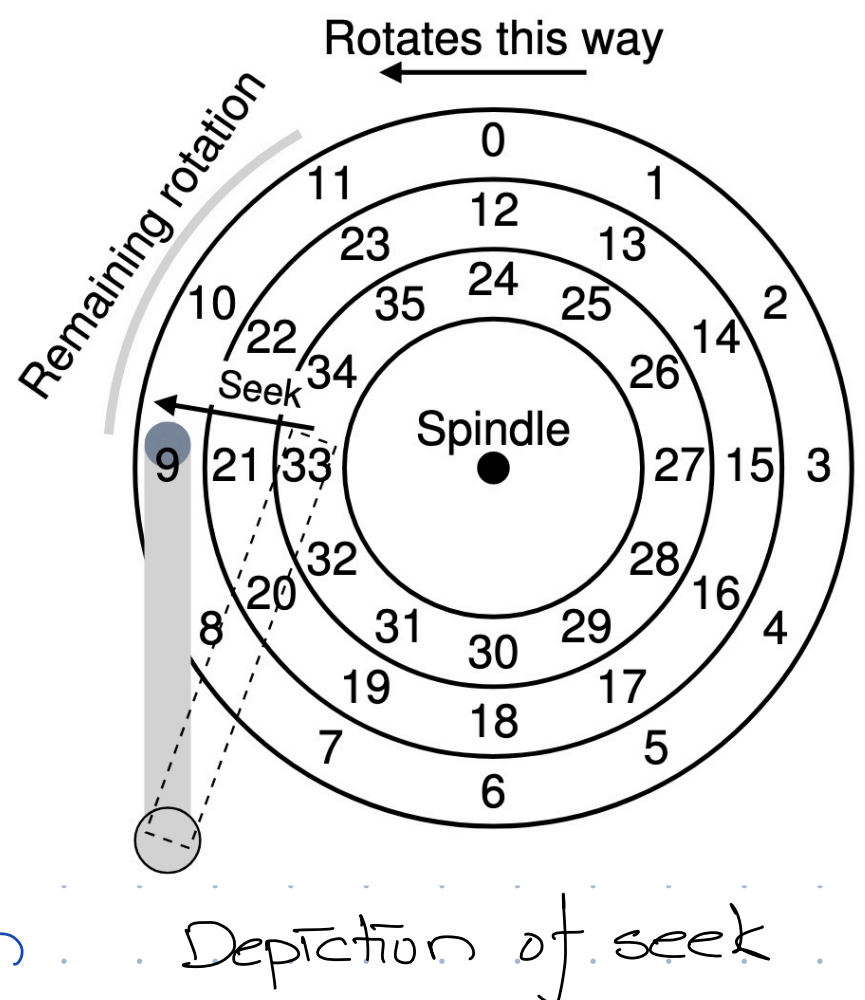
After seek, the disk arm is positioned the head over the

correct track.

As we can see (right figure), during seek, the platter has rotated about 3 sectors.

Thus, we must endure a short rotational delay to be over sector 11.

When sector 11 passes under the disk head, transfer takes place, where data is either read from or written to the surface.



Some Other Details

Many drives employ a kind of track skew to make sure that sequential reads can be properly serviced even when crossing track boundaries.

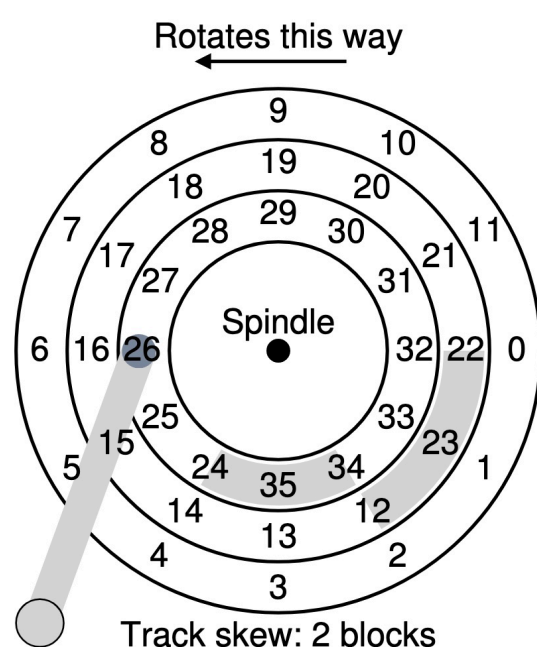


Figure 37.4: Three Tracks: Track Skew Of 2

Sectors are often skewed like this because when switching from one track to another, the disk needs time to reposition the head (even to neighboring tracks). Without such skew, the head would be moved to the next track but the desired next block would have already rotated under the head, and thus the drive would have to wait almost the entire rotational delay to access the next block.

Another reality is that outer tracks tend to have more sectors than inner tracks, which is a result of geometry; there is simply more room out there. These tracks are often referred to as **multi-zoned** disk drives, where the disk is organized into multiple zones, and where a zone is consecutive set of tracks on a surface. Each zone has the same number of sectors per track, and outer zones have more sectors than inner zones.

Finally, an important part of any modern disk drive is its **cache**, for historical reasons sometimes called a **track buffer**. This cache is just some small amount of memory (usually around 8 or 16 MB) which the drive can use to hold data read from or written to the disk. For example, when reading a sector from the disk, the drive might decide to read in all of the sectors on that track and cache them in its memory; doing so allows the drive to quickly respond to any subsequent requests to the same track.

On writes, the drive has a choice: should it acknowledge the write has completed when it has put the data in its memory, or after the write has actually been written to disk? The former is called **write back** caching (or sometimes **immediate reporting**), and the latter **write through**. Write back caching sometimes makes the drive appear “faster”, but can be dangerous; if the file system or applications require that data be written to disk in a certain order for correctness, write-back caching can lead to problems (read the chapter on file-system journaling for details).

ASIDE: DIMENSIONAL ANALYSIS

Remember in Chemistry class, how you solved virtually every problem by simply setting up the units such that they canceled out, and somehow the answers popped out as a result? That chemical magic is known by the highfalutin name of **dimensional analysis** and it turns out it is useful in computer systems analysis too.

Let’s do an example to see how dimensional analysis works and why it is useful. In this case, assume you have to figure out how long, in milliseconds, a single rotation of a disk takes. Unfortunately, you are given only the **RPM** of the disk, or **rotations per minute**. Let’s assume we’re talking about a 10K RPM disk (i.e., it rotates 10,000 times per minute). How do we set up the dimensional analysis so that we get time per rotation in milliseconds?

To do so, we start by putting the desired units on the left; in this case, we wish to obtain the time (in milliseconds) per rotation, so that is exactly what we write down: $\frac{\text{Time (ms)}}{1 \text{ Rotation}}$. We then write down everything we know, making sure to cancel units where possible. First, we obtain $\frac{1 \text{ minute}}{10,000 \text{ Rotations}}$ (keeping rotation on the bottom, as that’s where it is on the left), then transform minutes into seconds with $\frac{60 \text{ seconds}}{1 \text{ minute}}$, and then finally transform seconds in milliseconds with $\frac{1000 \text{ ms}}{1 \text{ second}}$. The final result is the following (with units nicely canceled):

$$\frac{\text{Time (ms)}}{1 \text{ Rot.}} = \frac{1 \cancel{\text{minute}}}{10,000 \text{ Rot.}} \cdot \frac{60 \cancel{\text{seconds}}}{1 \cancel{\text{minute}}} \cdot \frac{1000 \text{ ms}}{1 \cancel{\text{second}}} = \frac{60,000 \text{ ms}}{10,000 \text{ Rot.}} = \frac{6 \text{ ms}}{\text{Rotation}}$$

As you can see from this example, dimensional analysis makes what seems intuitive into a simple and repeatable process. Beyond the RPM calculation above, it comes in handy with I/O analysis regularly. For example, you will often be given the transfer rate of a disk, e.g., 100 MB/second, and then asked: how long does it take to transfer a 512 KB block (in milliseconds)? With dimensional analysis, it’s easy:

$$\frac{\text{Time (ms)}}{1 \text{ Request}} = \frac{512 \cancel{\text{KB}}}{1 \text{ Request}} \cdot \frac{1 \cancel{\text{MB}}}{1024 \cancel{\text{KB}}} \cdot \frac{1 \cancel{\text{second}}}{100 \cancel{\text{MB}}} \cdot \frac{1000 \text{ ms}}{1 \cancel{\text{second}}} = \frac{5 \text{ ms}}{\text{Request}}$$

I/O Time: Doing The Math

I/O time is as:

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

Rate of I/O is as:

$$R_{I/O} = \frac{\text{Size}_{\text{transfer}}}{T_{I/O}}$$

Assume two workloads:

1. Random workload issues small 4KB reads to random locations on the disk.
2. Sequential workload reads large number of sectors consecutively from the disk.

Further, we have two hard disks:

	"high performance" Cheetah 15K.5	"Capacity" Barracuda
Capacity	300 GB	1 TB
RPM	15,000	7,200
Average Seek	4 ms	9 ms
Max Transfer	125 MB/s	105 MB/s
Platters	4	4
Cache	16 MB	16/32 MB
Connects via	SCSI	SATA

Figure 37.5: Disk Drive Specs: SCSI Versus SATA

For workload 1 on cheetah:

$$T_{seek} = 4\text{ms (reported by manufacturer)}$$

$$T_{rotation} = 2\text{ms (calculated from RPM)}$$

$$\text{Time per rotation} = \frac{60}{1500}$$

$$= 4\text{ms}$$

On average, the disk will encounter a half rotation and thus 2ms is the average time

$$T_{\text{transfer}} = \frac{\text{Size}_{\text{transfer}}}{\text{Peak transfer rate}}$$

$$= \frac{4\text{KB}}{125\text{MB/s}}$$

$$= 30\text{microsecs}$$

For workload 2

We can assume that there is a single seek & rotation before a very long transfer.

For simplicity, assume size of transfer is 100MB

Cheetah

Barracuda

$$T_{I/O} = 800\text{ms}$$

$$T_{I/O} = 950\text{ms}$$

$$R_{I/O} = 125\text{MB/s}$$

$$R_{I/O} = 105\text{MB/s}$$

	Cheetah	Barracuda
$R_{I/O}$ Random	0.66 MB/s	0.31 MB/s
$R_{I/O}$ Sequential	125 MB/s	105 MB/s

Figure 37.6: Disk Drive Performance: SCSI Versus SATA

- There huge gap in drive performance b/w random & sequential workloads
- There is a large difference in performance b/w high end performance & low end capacity drives.

ASIDE: COMPUTING THE “AVERAGE” SEEK

In many books and papers, you will see average disk-seek time cited as being roughly one-third of the full seek time. Where does this come from?

Turns out it arises from a simple calculation based on average seek *distance*, not time. Imagine the disk as a set of tracks, from 0 to N . The seek distance between any two tracks x and y is thus computed as the absolute value of the difference between them: $|x - y|$.

To compute the average seek distance, all you need to do is to first add up all possible seek distances:

$$\sum_{x=0}^N \sum_{y=0}^N |x - y|. \quad (37.4)$$

Then, divide this by the number of different possible seeks: N^2 . To compute the sum, we'll just use the integral form:

$$\int_{x=0}^N \int_{y=0}^N |x - y| \, dy \, dx. \quad (37.5)$$

To compute the inner integral, let's break out the absolute value:

$$\int_{y=0}^x (x - y) \, dy + \int_{y=x}^N (y - x) \, dy. \quad (37.6)$$

Solving this leads to $(xy - \frac{1}{2}y^2)|_0^x + (\frac{1}{2}y^2 - xy)|_x^N$ which can be simplified to $(x^2 - Nx + \frac{1}{2}N^2)$. Now we have to compute the outer integral:

$$\int_{x=0}^N (x^2 - Nx + \frac{1}{2}N^2) \, dx, \quad (37.7)$$

which results in:

$$\left(\frac{1}{3}x^3 - \frac{N}{2}x^2 + \frac{N^2}{2}x \right) \Big|_0^N = \frac{N^3}{3}. \quad (37.8)$$

Remember that we still have to divide by the total number of seeks (N^2) to compute the average seek distance: $(\frac{N^3}{3})/(N^2) = \frac{1}{3}N$. Thus the average seek distance on a disk, over all possible seeks, is one-third the full distance. And now when you hear that an average seek is one-third of a full seek, you'll know where it came from.

Disk Scheduling

Given a set of I/O requests, the disk scheduler examines the requests & decides which one to schedule next.

In disk scheduling, we can approximate how long a disk request will take by estimating the seek and possible rotational delay of a request.

Thus, the disk scheduler will try to follow the principles of Shortest Job First by picking the request that will take least time to serve.

SSTF: Shortest Seek Time First

SSTF orders the queue of I/O requests by track, picking requests on the nearest track to complete first.

For example, assuming the current position of head is over the inner track, and we have requests for sectors 21 (middle track) & 2 (outer track).

SSTF will issue request to 21 first, wait for it to complete, and then issue the request to 2.

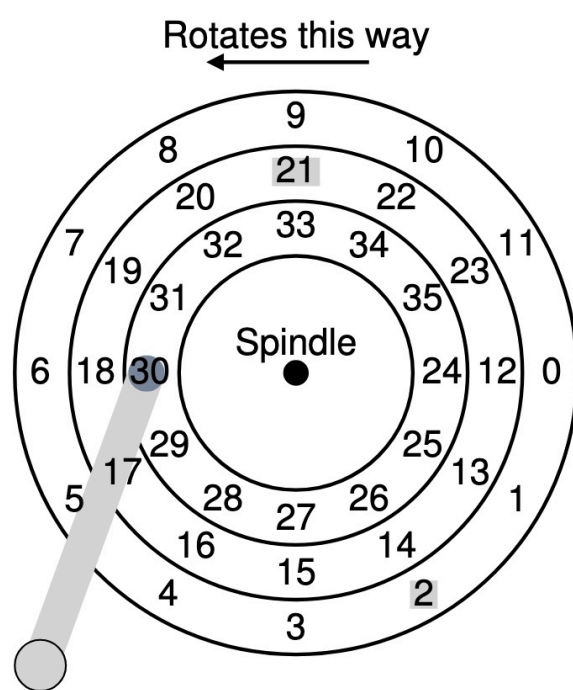


Figure 37.7: SSTF: Scheduling Requests 21 And 2

SSTF works well in this example but it not a panacea for following reasons:

1. The drive's geometry is not available to the

host OS. Rather, it sees an array of blocks

Sectors 1	Sectors 2							
Sectors n-1	Sectors n							

Thus, OS implements nearest-block first (NBF), which schedules the request with nearest block address.

2. Starvation: In above example, if there were a steady stream of requests to the inner track, where the head is currently positioned.

Requests to any other track would be ignored.

CRUX: HOW TO HANDLE DISK STARVATION

How can we implement SSTF-like scheduling but avoid starvation?

Elevator (a.k.a SCAN or C-SCAN)

This algorithm simply moves back & forth across the disk servicing requests in order across the tracks.

A single pass across the disk is called sweep.

Thus, if a request comes for a block on a track that has already been serviced on this sweep of the disk, it is not handled immediately, but rather queued until the next sweep (in the other direction).

In F-SCAN we have two queues, it works as:

- **Queue 1 (The Frozen/Active Queue):** This contains all the requests that were already waiting when the current sweep started. The drive "freezes" this list and promises to finish every single one of them before looking at anything new.
- **Queue 2 (The Incoming/Buffer Queue):** Any new request that arrives while the drive is busy working on Queue 1 gets thrown into this second queue. It doesn't matter if the new request is for a sector right next to the head; it has to wait in Queue 2.

C-SCAN is another common variant, short for **Circular SCAN**. Instead of sweeping in both directions across the disk, the algorithm only sweeps from outer-to-inner, and then resets at the outer track to begin again. Doing so is a bit more fair to inner and outer tracks, as pure back-and-forth SCAN favors the middle tracks, i.e., after servicing the outer track, SCAN passes through the middle twice before coming back to the

track, SCAN passes through the middle twice before coming back to the outer track again.

However, SCAN & SSTF ignore rotation.

CRUX: HOW TO ACCOUNT FOR DISK ROTATION COSTS

How can we implement an algorithm that more closely approximates SJF by taking *both* seek and rotation into account?

SPTF: Shortest Positioning Time First

Also known as shortest access-time first (SATF).

In the figure (Right), the head is positioned over sector 30 on the inner track.

The scheduler has to decide: should it schedule sector 16 or sector 8 for its next request?

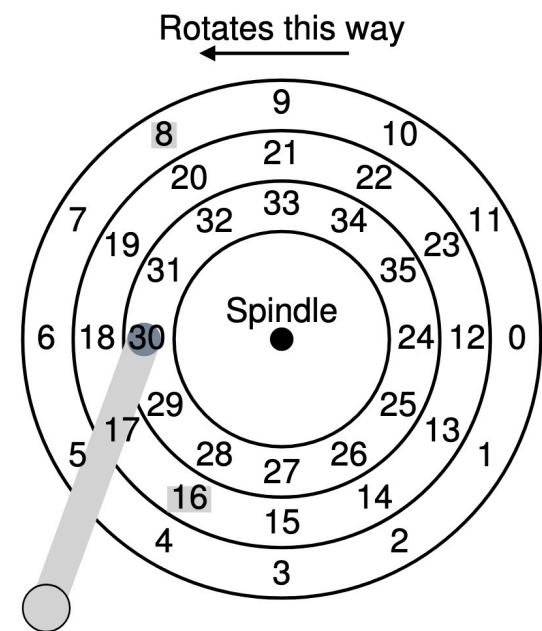


Figure 37.8: SSTF: Sometimes Not Good Enough

The answer depends on here is the relative time of seeking as compared to rotation. If, in our example, seek time is much higher than rotational delay, then SSTF (and variants) are just fine. However, imagine if seek is quite a bit faster than rotation. Then, in our example, it would make more sense to seek *further* to service request 8 on the outer track than it would to perform the shorter seek to the middle track to service 16, which has to rotate all the way around before passing under the disk head.

On modern drives, as we saw above, both seek and rotation are roughly equivalent (depending, of course, on the exact requests), and thus SPTF is useful and improves performance. However, it is even more difficult to implement in an OS, which generally does not have a good idea where track boundaries are or where the disk head currently is (in a rotational sense). Thus, SPTF is usually performed inside a drive, described below.

Other Scheduling Issues

Where is disk scheduling performed on modern systems?

In older systems, the OS did all the scheduling.

In modern systems, disks can accommodate

multiple outstanding requests, & have sophisticated internal schedulers themselves.

Thus, OS scheduler usually picks what it thinks the best few requests are (say 16) & issues them all to disk.

The disk then uses its internal knowledge of head position & detailed track layout information to service said requests.

Another important related task performed by disk schedulers is **I/O merging**. For example, imagine a series of requests to read blocks 33, then 8, then 34, as in Figure 37.8. In this case, the scheduler should **merge** the requests for blocks 33 and 34 into a single two-block request; any re-ordering that the scheduler does is performed upon the merged requests. Merging is particularly important at the OS level, as it reduces the number of requests sent to the disk and thus lowers overheads.

One final problem that modern schedulers address is this: how long should the system wait before issuing an I/O to disk? One might naively think that the disk, once it has even a single I/O, should immediately issue the request to the drive; this approach is called **work-conserving**, as the disk will never be idle if there are requests to serve. However, research on **anticipatory disk scheduling** has shown that sometimes it is better to wait for a bit [ID01], in what is called a **non-work-conserving** approach. By waiting, a new and "better" request may arrive at the disk, and thus overall efficiency is increased.